

RAPPORT DE PROJET

Test des Logiciels

Système de Gestion de Location de Voitures

Conception, Implémentation et Tests Complets

Étudiant	Rayen Bouyahia
Niveau	L2 – Classe CS01
Matière	Test des Logiciels
Année Universitaire	2025 – 2026

1. Introduction

Ce rapport présente le projet de gestion de location de voitures développé dans le cadre de la matière « Test des Logiciels » (L2CS01). L'objectif principal est de concevoir une application Python structurée selon les principes du développement piloté par les tests (TDD), et de la valider par une stratégie de tests à trois niveaux : tests unitaires, tests d'intégration et tests de comportement (BDD).

Le projet illustre comment tester rigoureusement un logiciel de gestion métier en isolant les composants, en vérifiant leurs interactions et en simulant des scénarios d'usage réels.

1.1 Objectifs du projet

- Concevoir un système de location de voitures orienté objet en Python
- Appliquer la démarche TDD (Test-Driven Development)
- Rédiger des tests unitaires, d'intégration et BDD avec le framework unittest
- Valider la robustesse du système face aux cas d'erreur et aux scénarios métier

1.2 Structure du projet

Le projet est organisé en huit fichiers Python aux responsabilités bien séparées :

Fichier	Rôle
voiture.py	Classe Voiture – modèle de données d'un véhicule
location.py	Classe Location – modèle de données d'une location
systeme_location.py	Classe SystemeLocation – moteur métier principal
main.py	Jeu d'essai complet et point d'entrée (démonstration)
test_unitaires.py	14 tests unitaires (TDD) sur Voiture et Location
test_integration.py	10 tests d'intégration sur SystemeLocation
test_bdd.py	6 scénarios BDD (Given / When / Then)
lancer_tous_les_tests.py	Runner global – rapport consolidé des 30 tests

2. Architecture du Système

2.1 Classe Voiture (voiture.py)

La classe Voiture représente un véhicule dans le parc de l'agence. Elle encapsule les données essentielles d'un véhicule et garantit leur validité dès la construction.

Attribut	Type	Description
immatriculation	str	Identifiant unique, normalisé en majuscules
marque	str	Marque du fabricant (ex : Renault, Peugeot)
modele	str	Modèle précis du véhicule
tarif_journalier	float	Tarif en Dinars Tunisiens – doit être > 0
disponible	bool	True = disponible, False = actuellement louée

Règles de validation appliquées dans le constructeur :

- L'immatriculation ne peut pas être vide ou ne contenir que des espaces
- Le tarif journalier doit être strictement positif (> 0)
- L'immatriculation est automatiquement mise en majuscules (strip().upper())

2.2 Classe Location (location.py)

La classe Location modélise une transaction de location. Elle lie un client, un véhicule, une date de début et une durée prévue. Elle intègre le calcul du coût total avec la gestion des pénalités de retard.

Élément	Description
PENALITE_PAR_JOUR	Constante de classe : 15 DT par jour de retard
date_retour_prevue	Calculée automatiquement : date_debut + timedelta(duree_prevue)
date_retour_reelle	Remplie lors du retour effectif de la voiture
terminee	Booléen – passe à True lors de l'enregistrement du retour
calculer_cout()	Retourne un dict : duree_reelle, cout_base, jours_retard, penalite, total

Formule de calcul du coût :

Coût de base = durée_réelle × tarif_journalier
Pénalité = max(0, jours_retard) × 15 DT
TOTAL = Coût de base + Pénalité

2.3 Classe SystemeLocation (systeme_location.py)

C'est le moteur central du système. Il coordonne l'ensemble des opérations de gestion de l'agence.

Méthode	Rôle
---------	------

ajouter_voiture()	Crée et enregistre une voiture ; lève ValueError si doublon
consulter_disponibilite()	Retourne la liste des voitures avec disponible=True
get_voiture(immat)	Cherche une voiture par immatriculation (None si absente)
louer_voiture()	Crée une Location, met disponible=False ; lève ValueError si indisponible
retourner_voiture()	Enregistre le retour, remet disponible=True, calcule le coût final
_trouver_location_active()	Méthode privée : cherche une location en cours pour une immat

3. Stratégie et Plan de Tests

Le projet adopte une approche pyramidale à trois niveaux, conforme aux bonnes pratiques du Test des Logiciels :

Niveau	Description	Nombre de tests	Fichier
Tests Unitaires	Vérifient chaque classe isolément (Voiture, Location)	14 tests	test_unitaires.py
Tests d'Intégration	Vérifient les interactions entre modules (SystemeLocation)	10 tests	test_integration.py
Tests BDD	Scénarios métier complets (Given / When / Then)	6 scénarios	test_bdd.py
TOTAL	Couverture complète du comportement attendu	30 tests	lancer_tous_les_tests.py

3.1 Tests Unitaires (test_unitaires.py)

Les tests unitaires suivent la démarche TDD : chaque test est écrit pour valider un comportement précis et isolé, sans dépendance externe au module testé.

Tests de la classe Voiture (7 tests) :

#	Nom du test	Comportement vérifié	Résultat
1	test_creation_voiture_valide	Attributs corrects + disponible=True par défaut	✓ PASS
2	test_immatriculation_mise_en_majuscule	Normalisation automatique en majuscules	✓ PASS
3	test_immatriculation_vide_leve_exception	ValueError si immatriculation vide	✓ PASS
4	test_tarif_negatif_leve_exception	ValueError si tarif < 0	✓ PASS
5	test_tarif_zero_leve_exception	ValueError si tarif = 0	✓ PASS
6	test_str_voiture_disponible	__str__ contient 'Disponible'	✓ PASS
7	test_str_voiture_louee	__str__ contient 'Louée' si disponible=False	✓ PASS

Tests de la classe Location (7 tests) :

#	Nom du test	Comportement vérifié	Résultat
8	test_creation_location_valide	Attributs et date_retour_prevue corrects	✓ PASS
9	test_cout_sans_retard	5j × 100 DT = 500 DT, pénalité = 0	✓ PASS
10	test_cout_avec_retard	3j retard : +3×15=45 DT, total=845 DT	✓ PASS
11	test_cout_retour_anticipe	Retour anticipé : pénalité = 0	✓ PASS
12	test_duree_nulle_leve_exception	ValueError si durée ≤ 0	✓ PASS

13	test_client_vide_leve_exception	ValueError si nom client vide	✓ PASS
14	test_calcul_cout_sans_date_retour_leve_exception	ValueError sans date_retour_reelle	✓ PASS

3.2 Tests d'Intégration (test_integration.py)

Les tests d'intégration vérifient que les trois classes (Voiture, Location, SystemeLocation) collaborent correctement pour réaliser les fonctionnalités métier de bout en bout.

#	Nom du test	Comportement vérifié	Résultat
1	test_ajout_voiture_et_consultation	3 voitures ajoutées → 3 disponibles	✓ PASS
2	test_ajout_voiture_dupliquee_leve_exception	Double immat → ValueError	✓ PASS
3	test_location_reduit_disponibilites	Après location → disponibilité passe de 3 à 2	✓ PASS
4	test_location_voiture_indisponible_leve_exception	Location d'une voiture louée → ValueError	✓ PASS
5	test_location_voiture_inexistante_leve_exception	Immat inconnue → ValueError	✓ PASS
6	test_retour_voiture_sans_retard	3j × 80 DT = 240 DT, voiture à nouveau dispo	✓ PASS
7	test_retour_voiture_avec_retard	2j retard : total = 660 DT	✓ PASS
8	test_retour_sans_location_active_leve_exception	Retour sans location active → ValueError	✓ PASS
9	test_flux_complet_location_retour_relocation	Cycle complet : location → retour → re-location	✓ PASS
10	test_plusieurs_locations_simultanees	2 voitures louées → 1 seule disponible	✓ PASS

3.3 Tests BDD – Behaviour Driven Development (test_bdd.py)

Les tests BDD expriment le comportement du système du point de vue de l'utilisateur final. Chaque scénario est structuré selon le patron GIVEN / WHEN / THEN, qui documente l'état initial, l'action déclenchée et le résultat attendu.

Scénario 1 : Location réussie d'une voiture disponible

GIVEN Une voiture Renault Clio (TN-010-AA) est disponible dans le système
WHEN Le client « Rami Trabelsi » loue cette voiture pour 4 jours à partir du 10/05/2024
THEN La location est créée, la voiture n'est plus disponible, la date de retour est le 14/05/2024

Scénario 2 : Location impossible d'une voiture déjà louée

GIVEN La voiture TN-020-BB est actuellement louée par Client A
WHEN Client B tente de louer la même voiture
THEN Une ValueError est levée avec le message mentionnant TN-020-BB

Scénario 3 : Retour à temps – aucune pénalité

GIVEN Une VW Golf (120 DT/j) louée pour 5 jours à partir du 01/06/2024
WHEN Le client retourne la voiture le 06/06/2024 (date prévue)
THEN Total = 600 DT, pénalité = 0, voiture disponible

Scénario 4 : Retour avec 3 jours de retard

GIVEN Renault Clio (80 DT/j) louée pour 3 jours à partir du 10/07/2024
WHEN Retour le 16/07/2024 (3 jours de retard)
THEN Durée réelle = 6j, base = 480 DT, pénalité = 45 DT, total = 525 DT

Scénario 5 : Consultation des voitures disponibles

GIVEN 3 voitures enregistrées, 2 sont actuellement louées
WHEN On consulte la disponibilité
THEN 1 seule voiture (TN-A03) est retournée comme disponible

Scénario 6 : Ajout d'une voiture avec tarif invalide

GIVEN Le système de location est initialisé
WHEN On tente d'ajouter une voiture Renault Kangoo avec tarif = -50 DT
THEN Une ValueError est levée et aucune voiture n'est ajoutée au système

4. Résultats des Tests

L'ensemble des 30 tests a été exécuté avec le runner global lancer_tous_les_tests.py. Tous les tests passent avec succès.

30 Tests exécutés	30 Tests réussis	0 Échecs / Erreurs
-----------------------------	----------------------------	------------------------------

Suite de tests	Tests	Réussis	Échecs	%
Tests Unitaires – Voiture	7	7	0	100%
Tests Unitaires – Location	7	7	0	100%
Tests d'Intégration	10	10	0	100%
Tests BDD	6	6	0	100%
TOTAL	30	30	0	100%

4.1 Cas d'erreur testés

Les tests couvrent également les cas aux limites et les erreurs attendues, garantissant la robustesse du système :

Cas d'erreur	Exception levée
Immatriculation vide ou espaces uniquement	ValueError
Tarif journalier ≤ 0	ValueError
Durée de location ≤ 0 jours	ValueError
Nom de client vide	ValueError
Location d'une voiture inexistante	ValueError
Location d'une voiture déjà louée	ValueError
Retour sans location active	ValueError
Calcul de coût sans date de retour définie	ValueError
Ajout d'une voiture avec immatriculation dupliquée	ValueError

5. Conclusion

Ce projet a permis de mettre en pratique l'ensemble des concepts fondamentaux du test des logiciels étudiés en L2CS01. Le développement du système de location de voitures a été conduit selon une démarche TDD rigoureuse, où chaque fonctionnalité a été validée avant d'être intégrée.

Les trois niveaux de tests mis en œuvre offrent une couverture complète :

- Les tests unitaires assurent la fiabilité de chaque composant en isolation
- Les tests d'intégration garantissent la cohérence des interactions entre modules
- Les tests BDD (GIVEN / WHEN / THEN) valident le comportement du système selon des scénarios métier réalistes

L'ensemble des 30 tests passe avec succès, démontrant la robustesse du système face à des données valides, invalides et des séquences d'opérations complexes (location, retour, re-location).

✓ **30 / 30 tests réussis — Taux de succès : 100 %**

Système validé conformément aux exigences de la matière Test des Logiciels